

SEMESTER PROJECT

Smart Campus Access Management & Analytics System

Course: Discrete Structures

Language: C++

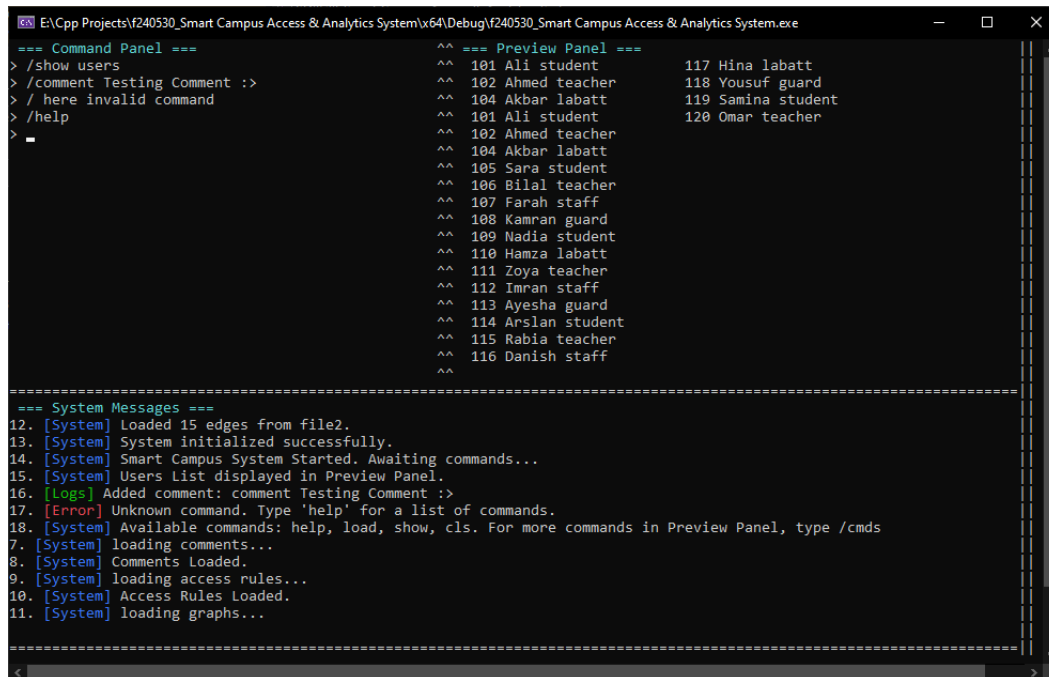
Semester: 3rd

Submitted by:

Abdullah Saleem

Roll No: 24F-0530

BS Computer Science – 3E



```
--- Command Panel ---
> /show users
> /comment Testing Comment :>
> / here invalid command
> /help
> -

--- Preview Panel ---
^^ 101 Ali student
^^ 102 Ahmed teacher
^^ 104 Akbar labatt
^^ 101 Ali student
^^ 102 Ahmed teacher
^^ 104 Akbar labatt
^^ 105 Sara student
^^ 106 Bilal teacher
^^ 107 Farah staff
^^ 108 Kamran guard
^^ 109 Nadia student
^^ 110 Hamza labatt
^^ 111 Zoya teacher
^^ 112 Imran staff
^^ 113 Ayesha guard
^^ 114 Anslan student
^^ 115 Rabia teacher
^^ 116 Danish staff
^^ 117 Hina labatt
^^ 118 Yousuf guard
^^ 119 Samina student
^^ 120 Omar teacher

--- System Messages ---
12. [System] Loaded 15 edges from file2.
13. [System] System initialized successfully.
14. [System] Smart Campus System Started. Awaiting commands...
15. [System] Users List displayed in Preview Panel.
16. [Logs] Added comment: comment Testing Comment :>
17. [Error] Unknown command. Type 'help' for a list of commands.
18. [System] Available commands: help, load, show, cls. For more commands in Preview Panel, type /cmds
7. [System] loading comments...
8. [System] Comments Loaded.
9. [System] loading access rules...
10. [System] Access Rules Loaded.
11. [System] loading graphs...
```

Abstract

This project implements a complete **Campus Access Management System** using concepts from **Discrete Structures**, **OOP**, **PF**, and **Data Structures**.

The system manages:

- **Users** (students, teachers, staff, admin)
- **Doors** (with ID, location, and type)
- **Access Rules** (Boolean conditions)
- **Access Logs & Comments**
- **Campus Graph** (nodes = rooms, edges = corridors)

Advanced functional components include:

- A **custom-built CLI Engine**
- A **Rule Engine** capable of evaluating Boolean expressions
- A **Graph module** implementing
 - **MST (Prim's Algorithm)**
 - **Euler Path Detection**
 - **Hamiltonian Path Search**

This system demonstrates the use of **graphs, sets, logic, expression evaluation, file handling, recursion, and algorithmic problem-solving**, fully aligned with the requirements of the Discrete Structures course.

1. Introduction

Modern campuses require strict access control to maintain security and manage movement across academic buildings.

The goal of this project is to simulate such a system using:

- C++ programming

- OOP class models
- File-based persistent storage
- Graph algorithms from Discrete Structures
- Boolean logic for rule evaluation

The system provides an efficient command-line interface and supports real-time access verification using both a **default access policy** and a **custom rule file**.

2. System Overview

The system is divided into five major modules:

1. **CLI Engine**
2. **User & Door Management**
3. **Rule Engine**
4. **Logging System (Access Logs & Comments)**
5. **Campus Graph Module**

Each module is implemented using a separate class, ensuring modularity and maintainability.

3. System Architecture

- **System**
 - Holds all users, doors, rules, logs, comments
 - Handles commands and acts as program controller
- **User**
 - Attributes: ID, name, role (student/teacher/staff/admin)
- **Door**
 - Attributes: ID, location, type (classroom/lab/datacenter/examhall/etc.)

- **AccessRule**
 - Stores rule ID and string
 - Evaluates rule using helper logic
- **AccessLog**
 - Records timestamp, user ID, door ID, and access result
- **Comment**
 - Simple user-created notes for system tracking
- **CampusGraph**
 - adj list representing graph
 - Functions for:
 - MST (Prim's)
 - Euler Path/Circuit detection
 - Hamiltonian Path search

4. CLI Engine

A custom command-line engine was developed to support:

- **Command Panel** (user input)
- **System Panel** (status messages)
- **Preview Panel** (data display)

Supported commands include:

- `/help` – list all commands
- `/load users`, `/load doors`, `/load rules`, `/load edges`
- `/show users`, `/show doors`, `/show rules`
- `/access <user> <door>`

- /logs, /comments
- /graph <display, mst, euler, hamilton>
- /cls, /systemcls, /previewcls, /restart

This engine is reusable for future C++ console projects.

5. User and Door Management

Users

Each user is defined by:

- ID
- Name
- Role

Users are loaded from a text file and stored in a vector.

Doors

Doors contain:

- ID
- Location
- Type

Also loaded from a file and stored in a vector.

6. Rule Engine

This is the most important logical component of the project.

Key Features

- Reads rules from rules.txt

- Supports boolean expressions with:
 - AND, OR, NOT
 - Parentheses
- Supports condition tokens:
 - role=student
 - type=lab
 - door=G1

Evaluation Steps

1. Replace tokens with boolean values (true/false)
2. Parse final expression using:
 - OR (lowest precedence)
 - AND
 - NOT
 - Parentheses recursively
3. If a rule returns **true**, access is **denied** and logged.

This approach mimics actual policy engines used in enterprise systems.

7. Logging System and Comments

Two log types:

1) Access Logs

Each access attempt records:

- Timestamp
- User ID

- Door ID
- Allowed / Denied

Stored in logs.txt.

Example log entry:

2025-02-25 14:21:05 | User 3 | Door 8 | Denied

2) Comments

A user-driven note system.

Simple lines of text, stored in comments.txt.

using /command <text> or #<text>

8. Campus Graph Module

9. Campus Graph Module

The graph represents campus buildings (nodes) and corridors (edges).

Edges are loaded from edges_euler.txt.

Graph Functions Implemented

8.1 Minimum Spanning Tree (Prim's Algorithm)

Used for planning network cable layout.

Outputs:

- All MST edges
- Total cost

8.2 Euler Path / Euler Circuit Detection

Verifies if:

- All vertices have even degree = Circuit
- Exactly two have odd degree = Path

Constructs Euler path using Hierholzer's Algorithm.

8.3 Hamiltonian Path Search

Implemented using backtracking.

Finds a path visiting every building exactly once.

This connects project theory directly to Discrete Structures concepts.

9. File Structure

- main.cpp
 - System.h
 - UserAndDoors.h
 - AccessRules.h
 - LogsAndComments.h
 - CampusGraph.h
 - ConsoleAndBufferEssentials.h
 - users.txt
 - doors.txt
 - rules.txt
 - edges.txt
 - edges2.txt (a backup set of edges, contains Euler path)
 - accesslogs.txt
 - comments.txt

10. Conclusion

This project successfully demonstrates the application of **Discrete Structures** in a real-world inspired system.

Key outcomes include:

- Understanding of graphs, logic, and rules
- Implementation of advanced parsing mechanisms
- Use of standard algorithms (Prim, Euler, Hamiltonian)
- Modular OOP design with reusable CLI engine
- File-based data persistence
- Correct handling of access policies and logging

This system can be extended into a fully functional campus security management tool.

Future Work

Planning on completing the CLI Engine template for future use. It was exciting to accidentally building something entirely new while working on this project.

Screenshot

```
E:\Cpp Projects\Smart Campus Access & Analytics System\Debug\Smart Campus Access & Analytics System.exe

=== Command Panel ===
> /show users
> /comment Testing Comment :>
> / here invalid command
> /help
> _

=== Preview Panel ===
^^ 101 Ali student      117 Hina labatt
^^ 102 Ahmed teacher   118 Yousuf guard
^^ 104 Akbar labatt    119 Samina student
^^ 101 Ali student     120 Omar teacher
^^ 102 Ahmed teacher
^^ 104 Akbar labatt
^^ 105 Sara student
^^ 106 Bilal teacher
^^ 107 Farah staff
^^ 108 Kamran guard
^^ 109 Nadia student
^^ 110 Hamza labatt
^^ 111 Zoya teacher
^^ 112 Imran staff
^^ 113 Ayesha guard
^^ 114 Arslan student
^^ 115 Rabia teacher
^^ 116 Danish staff
^^

=====
=== System Messages ===
12. [System] Loaded 15 edges from file2.
13. [System] System initialized successfully.
14. [System] Smart Campus System Started. Awaiting commands...
15. [System] Users List displayed in Preview Panel.
16. [Logs] Added comment: comment Testing Comment :>
17. [Error] Unknown command. Type 'help' for a list of commands.
18. [System] Available commands: help, load, show, cls. For more commands in Preview Panel, type /cmds
19. [System] loading comments...
20. [System] Comments Loaded.
21. [System] loading access rules...
22. [System] Access Rules Loaded.
23. [System] loading graphs...
```

'Till next time...